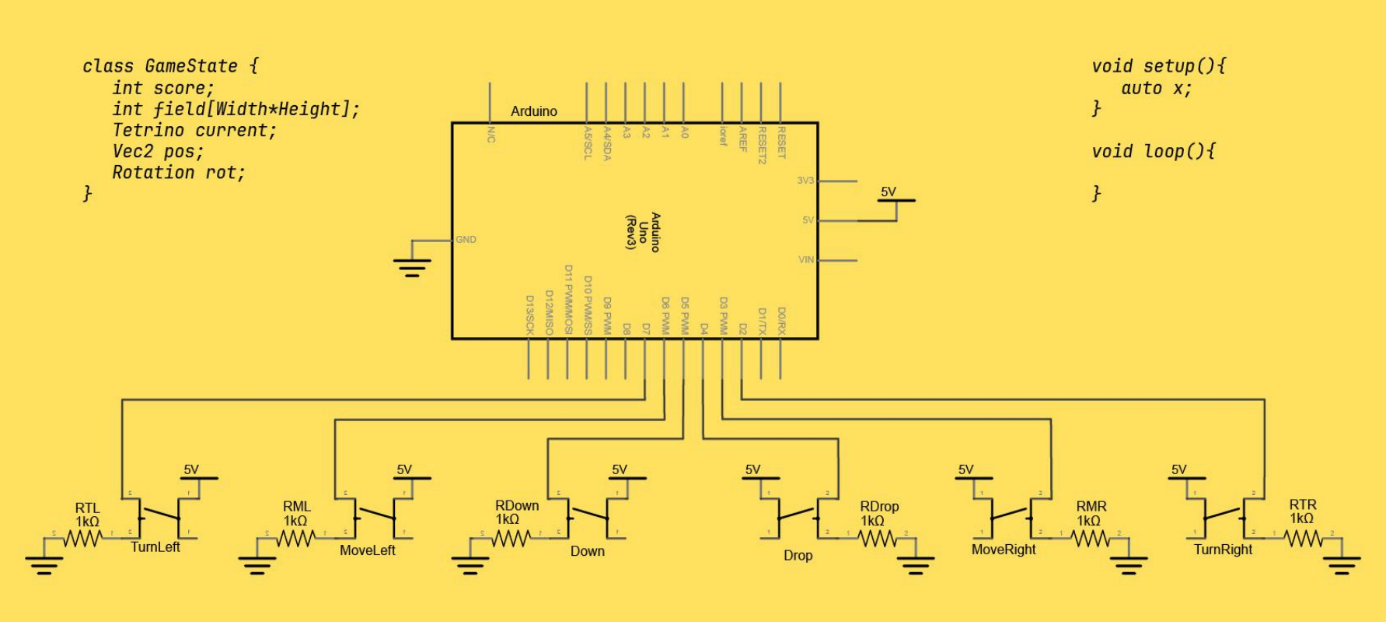
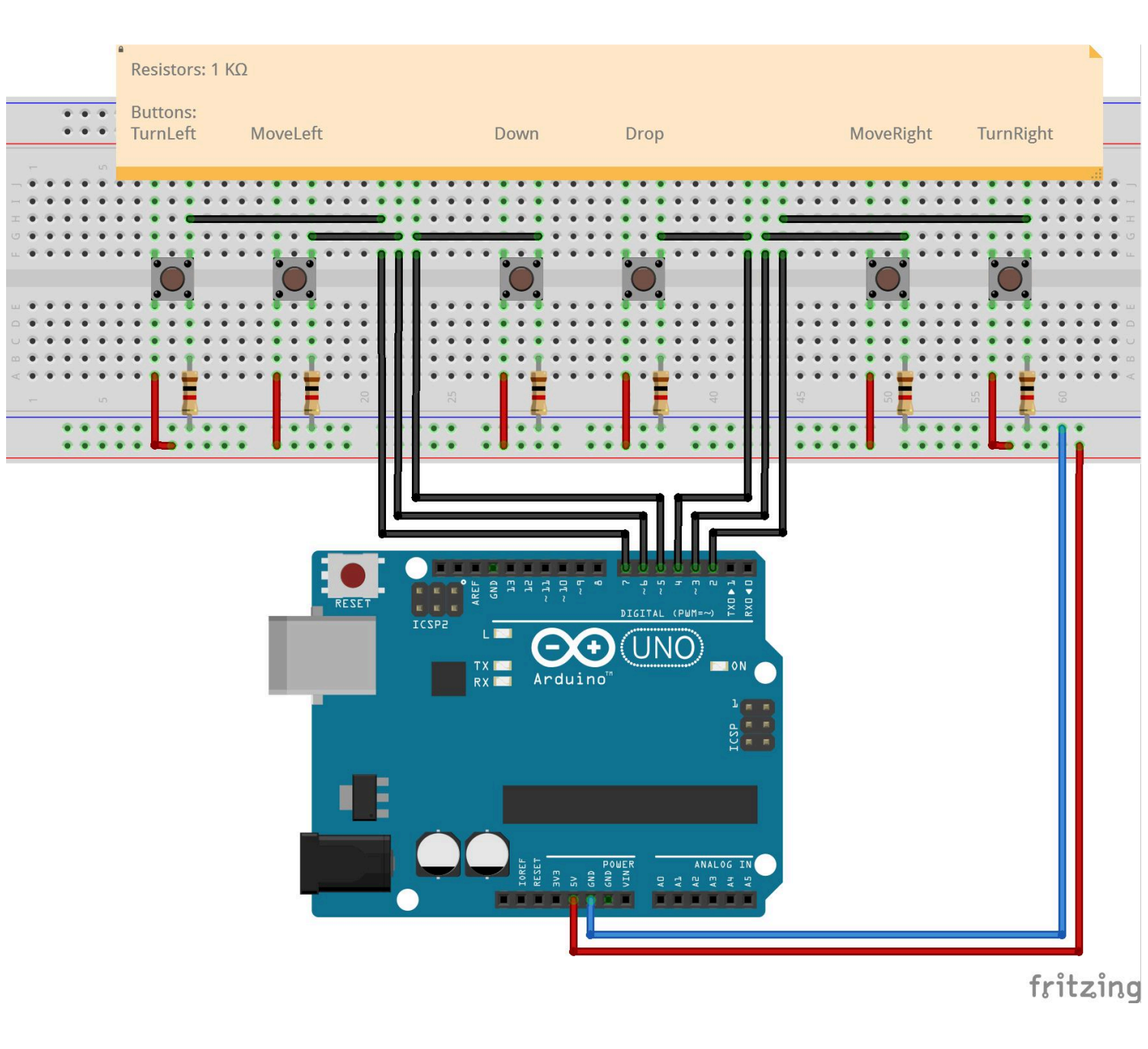
Embedded Programming

Arduino Tetris



Studienmodul	Technick der Mensch Maschiene Interaktion
Zeitraum	Oktober 2024 - Februar 2025
Team	Einzelarbeit
Tools	Ai Ps Pr Xd
Hochschule	Technische Hochschule Ingolstadt



Im Rahmen eines Hackathons an der Hochschule habe ich eine minimalistische, aber voll spielbare Version von Tetris für den Arduino entwickelt. Die erste Version entstand auf dem Arduino Uno R4, später folgte die Portierung auf den Uno Rev3, was einige technische Änderungen nötig machte.

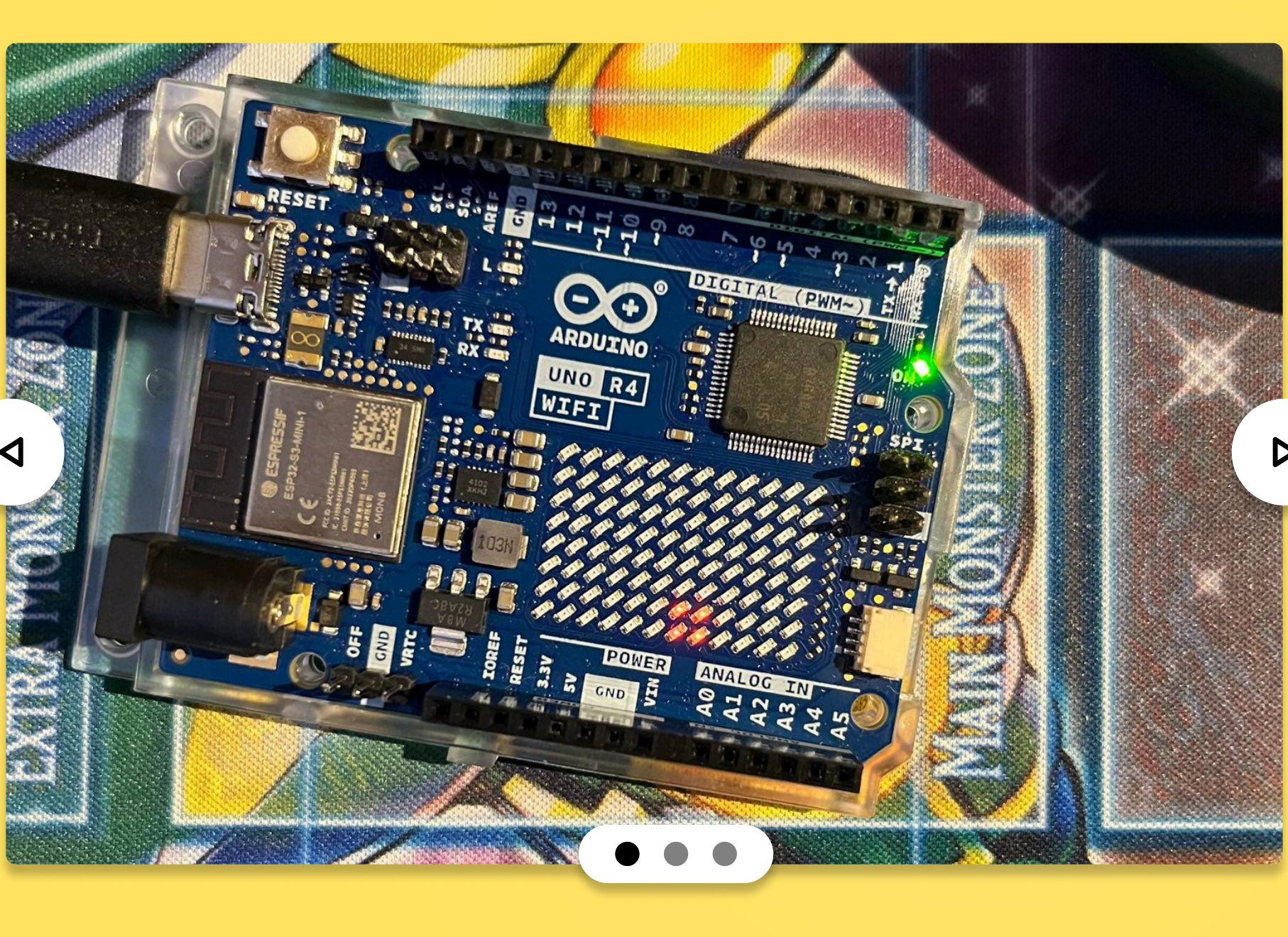
Hardware & Vorbereitung

Vor dem Hackathon habe ich nach Hardware gesucht und diese anschließend auch gekauft. Ziel war es, Tetris auf der im R4 eingebauten Matrix zu testen und dann auf einer LED-Matrix auszugeben. Leider war mir nicht bewusst, dass der R4 nicht auf demselben Chipdesign basiert, wie die vorherigen Arduinos, und daher nicht mit der Waveshare-Matrix kompatibel ist. Außerdem stellte sich heraus, dass der gegebene R3 zu schwach ist.

Die verwendete Hardware bestand aus folgenden teilen

- Arduino UNO R4 WIFI ()
- Arduino UNO R3 (gegeben vom Hackathon & Optional)
- Breadboard klein (Optional)
- Widerstände (min 6x 1kOhm)
- Kabel, Buttons & Breadboard groß
- WaveShare screen (Fehlentscheidung)

Ergebnis



Am Ende des Hackathons hatte ich ein funktionsfähiges Tetris Spiel auf der eingebauten Matrix des Arduino R4, jedoch nicht ohne Abzüge. Leider waren weder die mitgebrachte noch die gestellte LED-Matrix mit dem neuen R4 kompatibel und ich musste deswegen bei der eingebauten Matrix bleiben. Auch ein mit Matrix kompatibler Arduino R3 wurde gestellt, doch dieser war leider mit meinem, auf dem R4 getestetem, Source-Code nicht kompatibel (z.B wegen std::array).

Lessons Learned

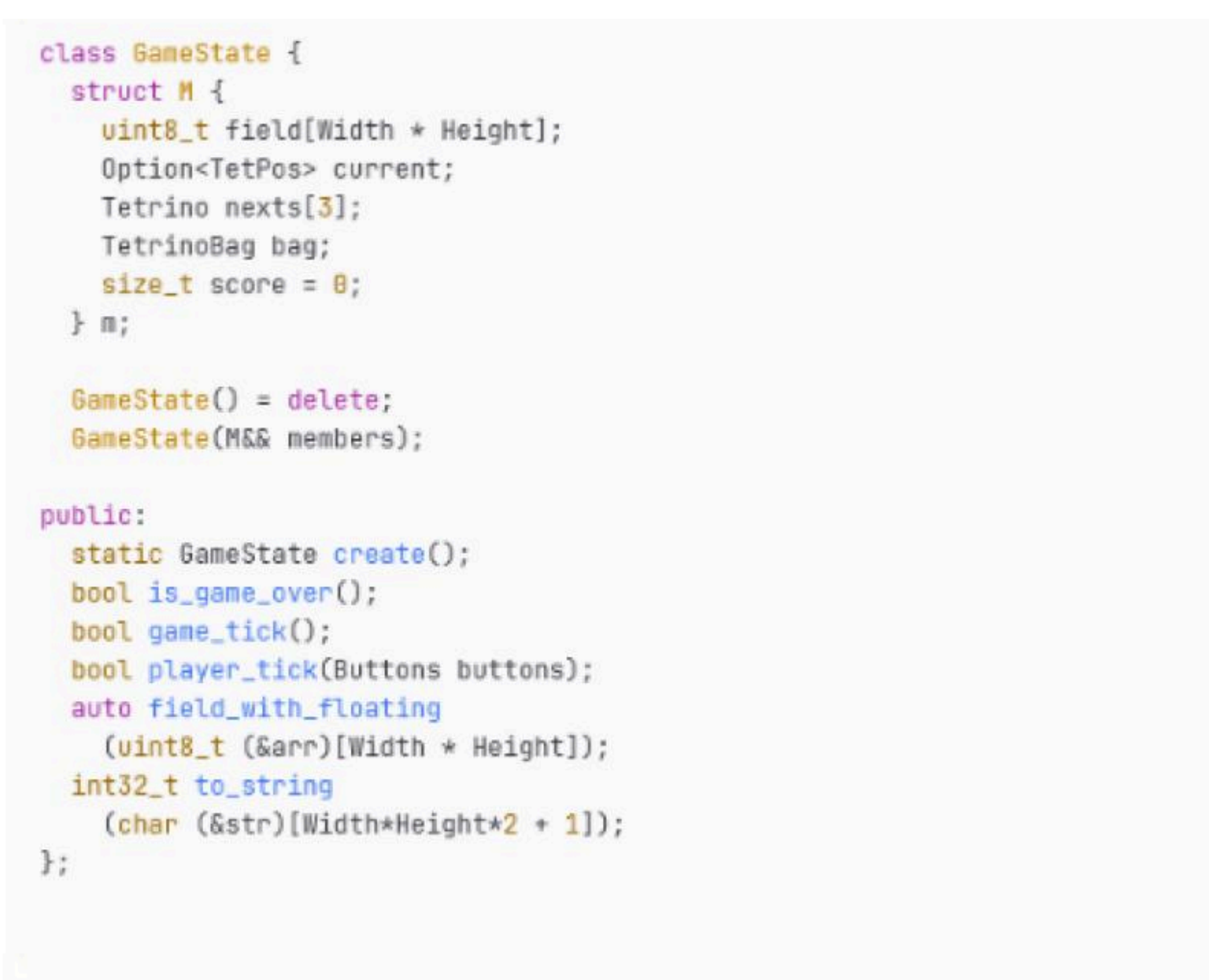
Ohne Betriebssystem steht deutlich weniger Unterstützung zur Verfügung. Die Kompatibilität mit der STL variiert stark. Die Arbeit mit Hardware erfordert oft Fingerspitzengefühl; Kurzschlüsse oder Verwirrung sind schnell möglich.

Follow-up work

Nach dem Hackathon habe ich die Tetris-Bibliothek vollständig überarbeitet, sodass sie nun auch auf Rev3-Arduinos funktioniert. Dabei ersetzte ich die bisher verwendete C++-STL durch klassischen C/C++-Code. Beispielsweise wurde std::array<T,N> durch einfache Array-Deklarationen wie type_T name[N] ersetzt. Zudem habe ich den bisherigen GameLoop, der über die Funktion tick() lief, in zwei getrennte Funktionen aufgeteilt: bool game_tick(); und bool player_tick(Buttons buttons);. Dadurch konnte ich die Eingabeverarbeitung vom automatischen Fallverhalten entkoppeln, was das Spiel deutlich reaktionsschneller macht. Eingaben können nun jederzeit erkannt werden, nicht nur während eines Ticks.

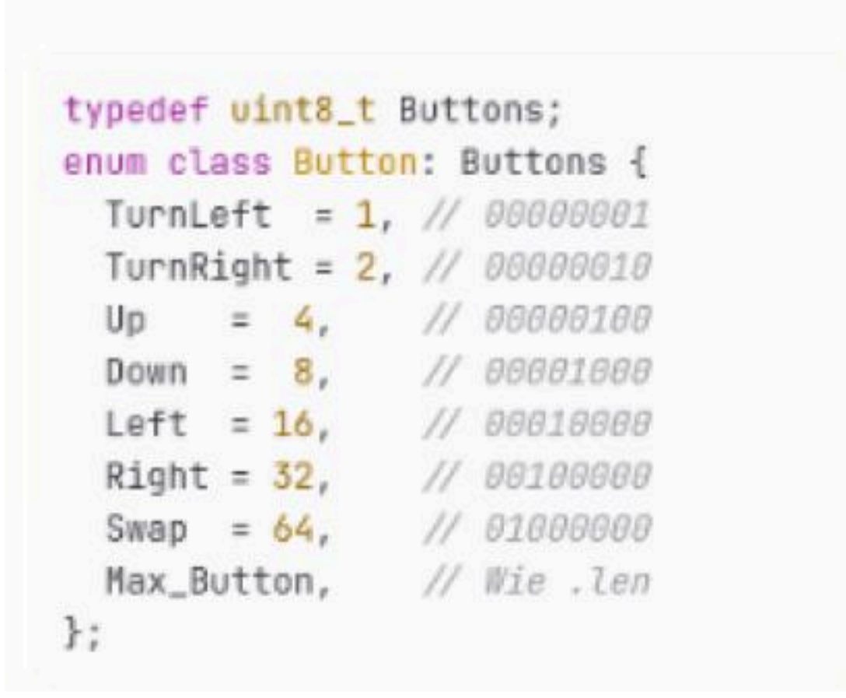
Umsetzung - Code

Für den Hackathon, habe ich eine Tetris Bibliothek in C++ entwickelt und als Nachbereitung auf R3 erweitert. Diese basiert auf GameState welches alle wichtigen Daten einer runde speichert und Tick-Methoden bereitstellt um das spiel zu treiben. Es stellt auch eine Methode bereit, um das aktuelle Feld entweder als String oder Liste zu bekommen und es anzeigen zu können.



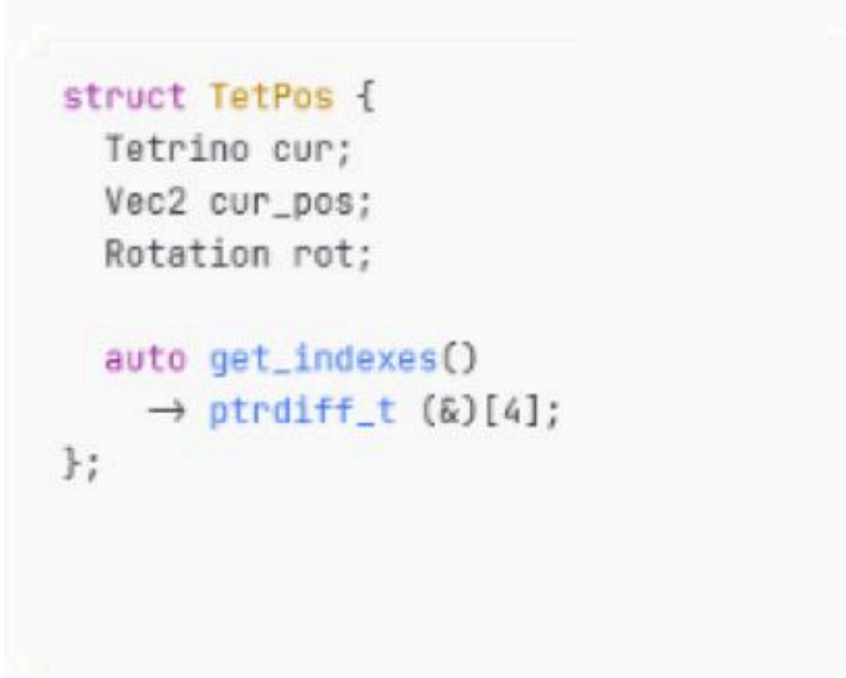
```
class GameState {
  struct M {
    uint8_t field[Width * Height];
    Option<TetPos> current;
    Tetrino nexts[3];
    TetrinoBag bag;
    size_t score = 0;
  } m;

  GameState() = delete;
  GameState(M&& members);

public:
  static GameState create();
  bool is_game_over();
  bool game_tick();
  bool player_tick(Buttons buttons);
  auto field_with_floating
    (uint8_t (&arr)[Width * Height]);
  int32_t to_string
    (char (&str)[Width*Height*2 + 1]);
};
```

Um die Nutzereingabe zu lesen werden 2 Typen exportiert: Buttons und Button. Button ist ein Enum welches alle Knöpfe auflistet und je einem Bit in einem Byte zuordnet, sodass alle möglichen Eingaben gleichzeitig und speichersparend verarbeitet werden können, da sie nun in einen Byte (Buttons) passen. Das ermöglicht schnelle Abfragen und kompakte Logik. Man kann sich dieses Flaggen-System vorstellen wie 8 boolesche Werte in einer Variable. Man kann diese Werte mit Bit shifts (<<) und Bit-Oder (|) setzen und mit Bit shifts und Bit-Und (&) lesen. (Mehr Dazu)

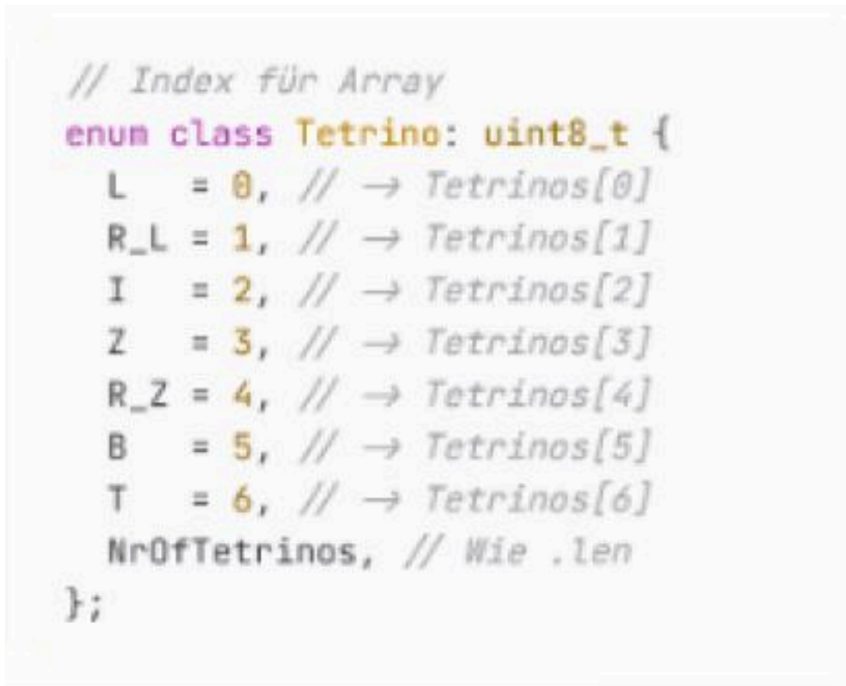


```
typedef uint8_t Buttons;
enum class Button: Buttons {
  TurnLeft  = 1, // 00000001
  TurnRight = 2, // 00000010
  Up    =  4,    // 00000100
  Down  =  8,    // 00001000
  Left  = 16,    // 00010000
  Right = 32,    // 00100000
  Swap  = 64,    // 01000000
  Max_Button,    // Wie .len
};
```

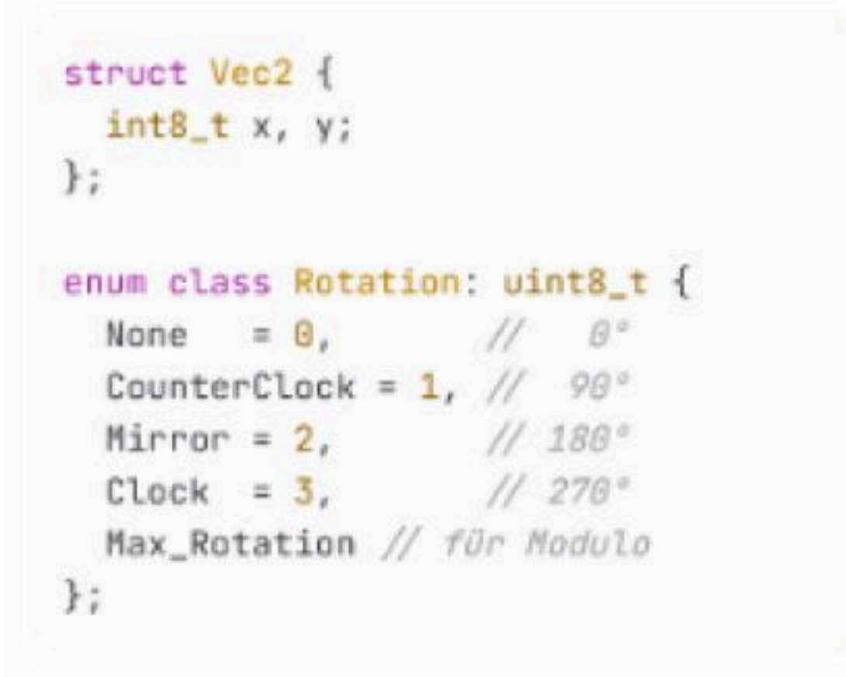
Die einzelnen Tetrinos sind in einem Enum als Indexe zu einem Array, welcher Postions-Matrizen der Tetrinos speichert. Um auch Position & Rotation zu speicher wird TetPos benutzt.



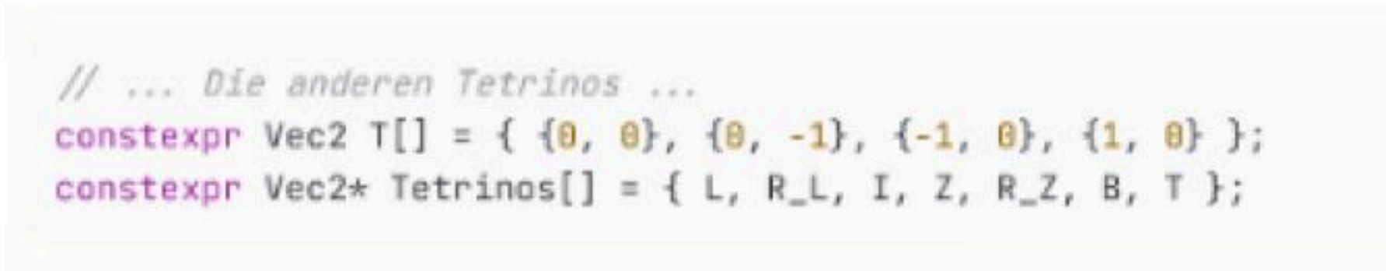
```
struct TetPos {
  Tetrino cur;
  Vec2 cur_pos;
  Rotation rot;

  auto get_indexes()
    → ptrdiff_t (&)[4];
};
```



```
// Index für Array
enum class Tetrino: uint8_t {
  L   = 0, // → Tetrinos[0]
  R_L = 1, // → Tetrinos[1]
  I   = 2, // → Tetrinos[2]
  Z   = 3, // → Tetrinos[3]
  R_Z = 4, // → Tetrinos[4]
  B   = 5, // → Tetrinos[5]
  T   = 6, // → Tetrinos[6]
  NrOfTetrinos, // Wie .len
};
```

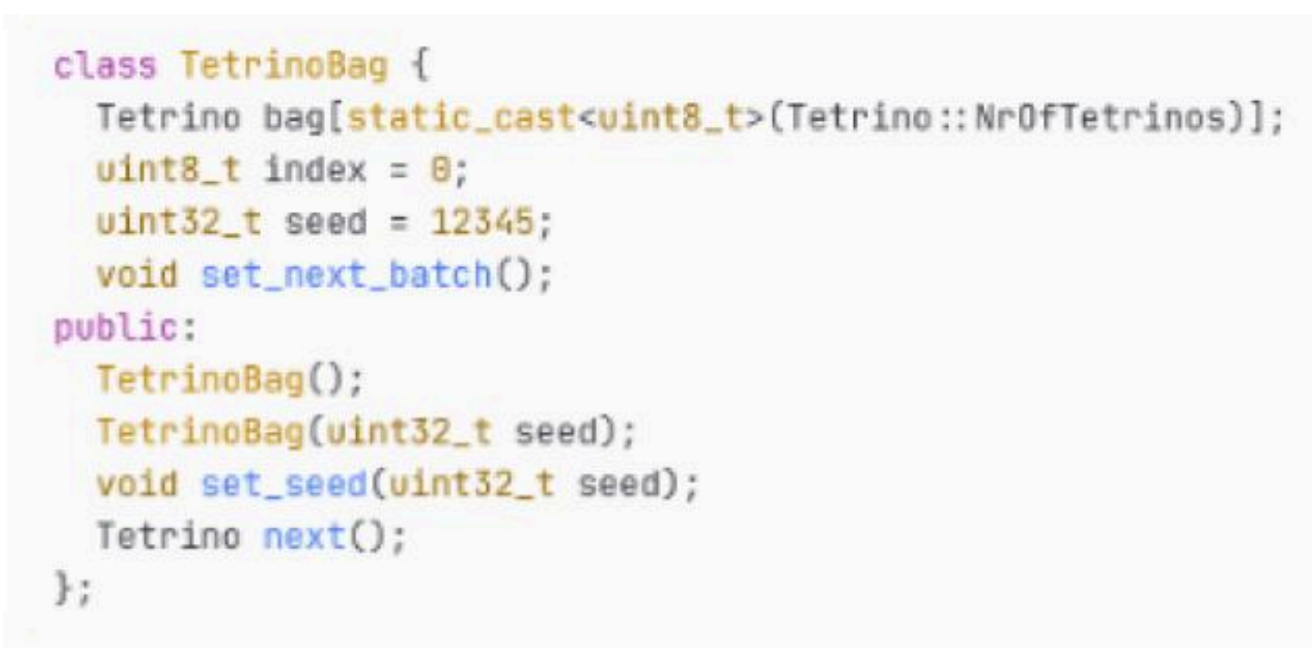


```
struct Vec2 {
  int8_t x, y;
};

enum class Rotation: uint8_t {
  None    = 0,       //   0°
  CounterClock = 1, //  90°
  Mirror = 2,        // 180°
  Clock   = 3,       // 270°
  Max_Rotation // für Modulo
};
```

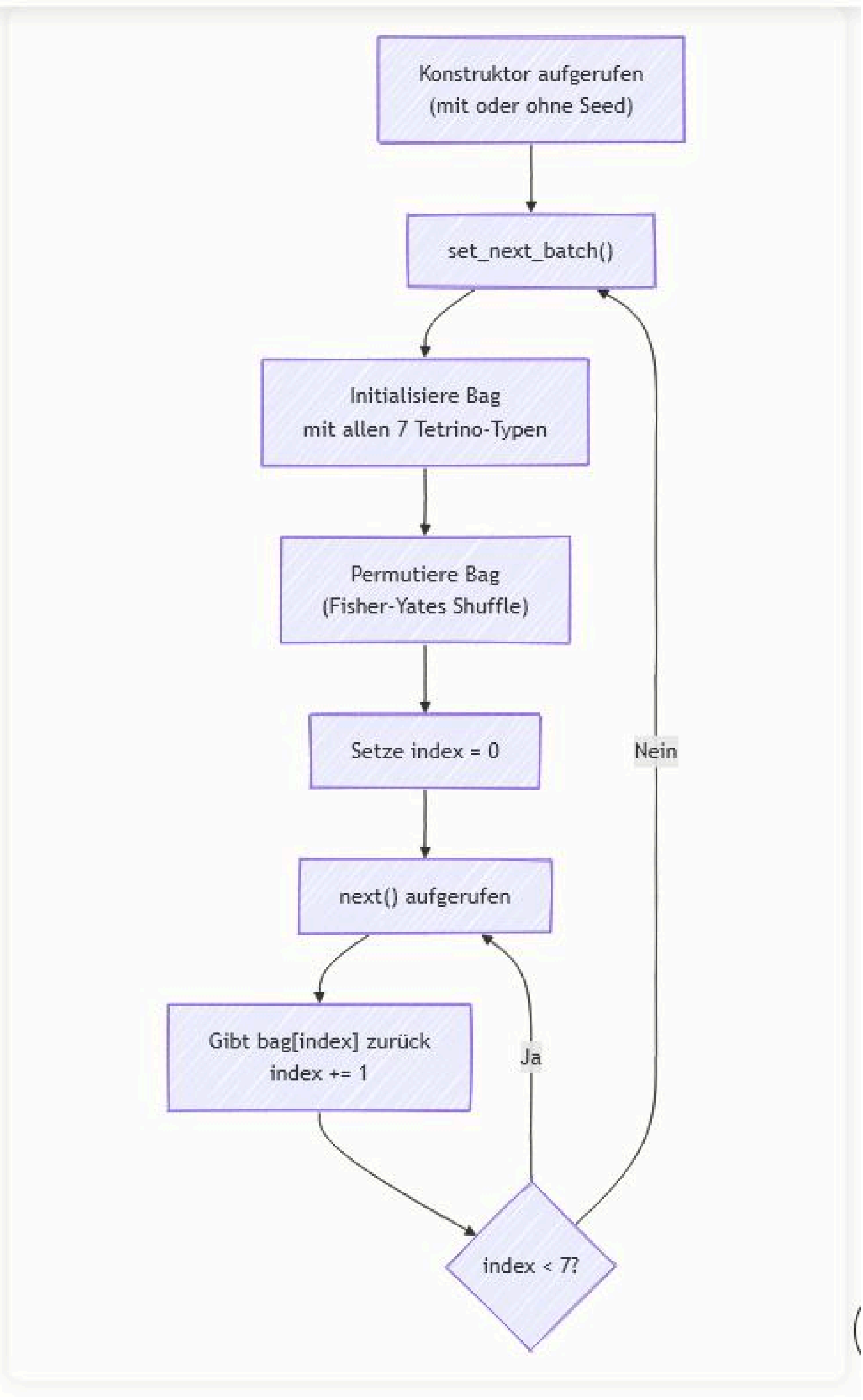


```
// ... Die anderen Tetrinos ...
constexpr Vec2 T[] = { {0, 0}, {0, -1}, {-1, 0}, {1, 0} };
constexpr Vec2* Tetrinos[] = { L, R_L, I, Z, R_Z, B, T };
```

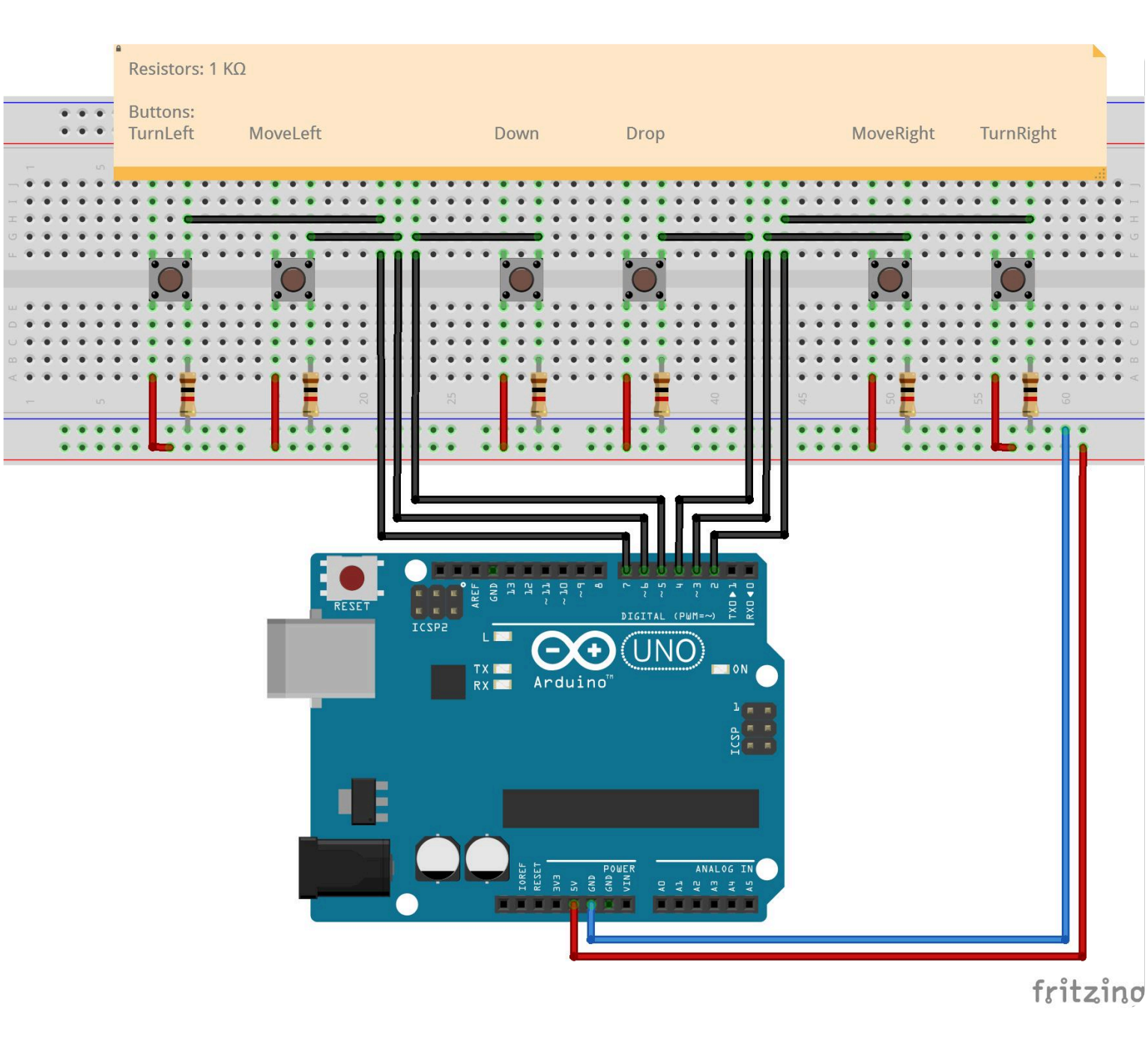
Die Erscheinungsraten der Tetrominos werden mithilfe eines Taschensystems gesteuert. Dabei werden alle sieben Tetrominos zunächst gesammelt, in zufälliger Reihenfolge angeordnet und anschließend nacheinander gezogen. So wird sichergestellt, dass jeder Tetromino pro Sequenz genau einmal vorkommt und keine Form überrepräsentiert ist, aber die Reihenfolge noch immer zufällig ist.



```
class TetrinoBag {
  Tetrino bag[static_cast<uint8_t>(Tetrino::NrOfTetrinos)];
  uint8_t index = 0;
  uint32_t seed = 12345;
  void set_next_batch();
public:
  TetrinoBag();
  TetrinoBag(uint32_t seed);
  void set_seed(uint32_t seed);
  Tetrino next();
};
```

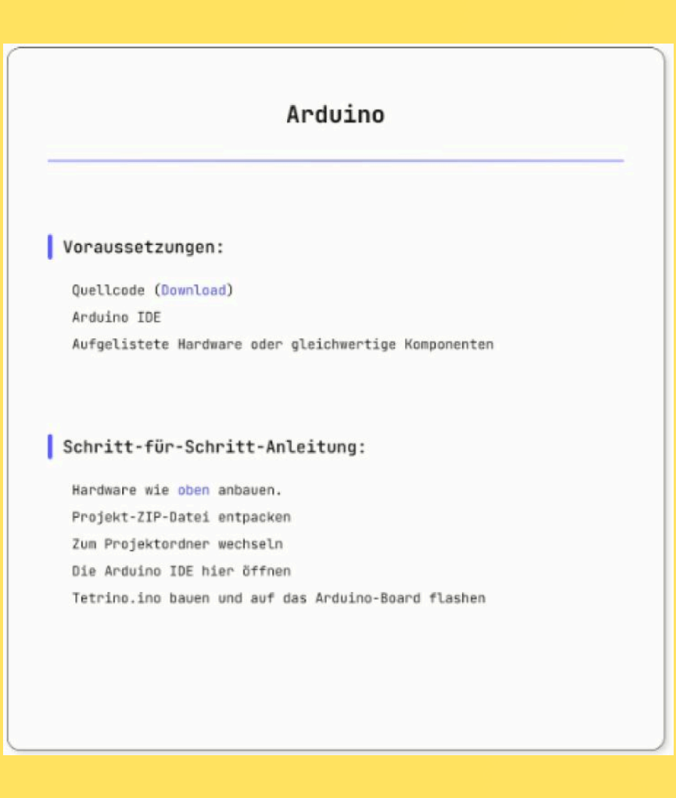


Umsetzung - Hardware



TODO

How to Run



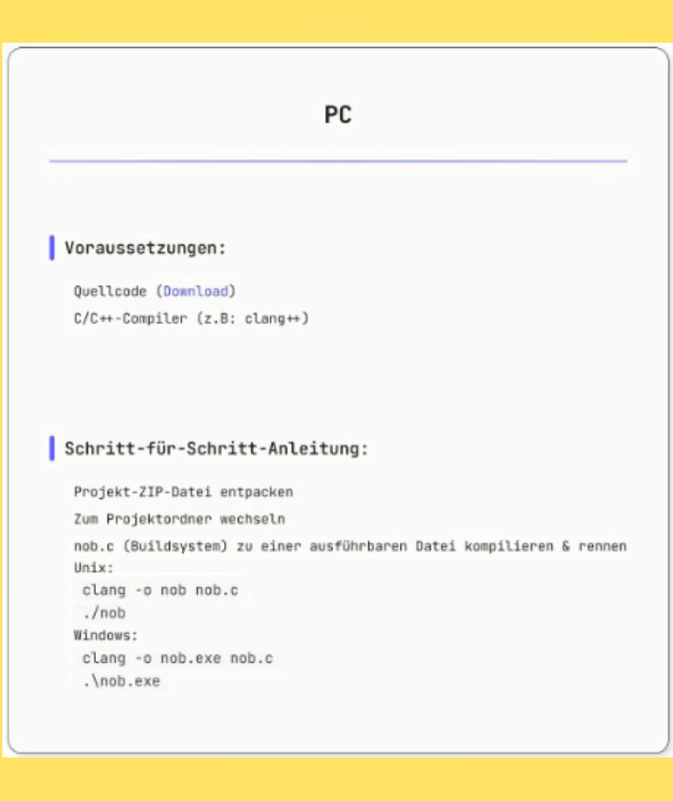
Arduino

Voraussetzungen:

Quellcode (Download)
Arduino IDE
Aufgelistete Hardware oder gleichwertige Komponenten

Schritt-für-Schritt-Anleitung:

Hardware wie oben anbauen.
Projekt-ZIP-Datei entpacken
Zum Projektordner wechseln
Die Arduino IDE hier öffnen
Tetrino.ino bauen und auf das Arduino-Board flashen



PC

Voraussetzungen:

Quellcode (Download)
C/C++-Compiler (z.B: clang++)

Schritt-für-Schritt-Anleitung:

Projekt-ZIP-Datei entpacken
Zum Projektordner wechseln
nob.c (Buildsystem) zu einer ausführbaren Datei kompilieren & rennen

```
Unix:
 clang -o nob nob.c
 ./nob
Windows:
 clang -o nob.exe nob.c
 .\nob.exe
```

CC BY-NY-ND © Tim Ruland 2026